# David Londoño

Pereira, Colombia (UTC-5) | +57 317 2939347 | dlondon.dev@gmail.com | linkedin.com/in/davidldv | github.com/davidldv | davidlondon.dev

AI Full-Stack Engineer at KitchenSync, working across React, Node.js and PostgreSQL on a back-office platform for restaurants. I work with coding agents from a written architecture and spec rather than from prompts. Two projects show how: **authzscan**, an agent pipeline that reviews Next.js codebases and publishes its own false-positive rate, and **Ghost AI**, a canvas that turns a system design into a Markdown spec. English C1, Spanish native, German B1. Available full time, remote (LATAM/EU/US) or relocation.

## Work Experience

**KitchenSync** **Oct 2026 – Present**
*Full-Stack Engineer (Contract)* *Remote*

- Build **Kit**, a back-office platform for restaurants covering accounting, payroll and accounts payable, across the TypeScript stack: **React** front ends, **Node.js** services and **PostgreSQL**.
- Unifying point-of-sale providers behind one validated **adapter layer**, and moving background syncs onto a durable, **idempotent job queue**.
- Use **AI coding agents** daily, scoping their work and reviewing and steering what they produce, and ship through small, **CI-gated** pull requests with safe schema migrations.

**Tambora** **Jul 2025 – Sep 2025**
*Frontend Developer (Contract)* *Remote*

- Read through a legacy jQuery codebase and migrated its business-critical modules to **React**. Bundle size dropped **40%** on the rebuilt surfaces, and the ad hoc DOM string injection went with it.
- Replaced manual SSH deploys with **Azure CI/CD** pipelines and put test gates in front of every release. A deploy went from over **two hours to under fifteen minutes**.

## Projects

**Ghost AI** | *Live* | *Code* | *Writeup* **Next.js** | **Liveblocks** | **AI SDK**
*Real-time architecture canvas: a team describes a system, an agent lays it out, the graph exports as a spec.*

- Multiplayer canvas on **Liveblocks** and **React Flow**, with nodes and edges stored in Postgres through **Prisma**. Prompts go through the **Vercel AI SDK** and come back as structured graph mutations, the same contract the UI uses, and the finished graph exports as a **Markdown spec** the next agent in the pipeline reads.
- Built the codebase **spec-first**. Four living context files (project overview, architecture and invariants, code standards, progress tracker) get read by every agent before it writes anything, and I scope and review each feature against them before implementation.

**authzscan** | *Code* | *Writeup* **TypeScript** | **ts-morph** | **SARIF**
*Agents that review an existing Next.js codebase for authorization bugs, with published accuracy.*

- Four phases. A deterministic **ts-morph** pass inventories every route handler, Server Action and auth idiom in the repo. A trace agent follows each client-controlled identifier to the query it reaches. A second, adversarial agent re-reads what trace found and kills its false positives. Results come out as **SARIF**, Markdown or JSON with exit codes CI can gate on.
- Measured on a seeded benchmark: **16/16** planted bugs found and **0/6** hardened twins flagged, at $2.10 and 19 minutes per scan. An oracle runner checks that the harness scores correctly no matter how the model performs. I then ran it on a real open-source repo, kept 1 genuine bug out of 11 candidates, and fixed the seven tool bugs the benchmark had hidden in **v0.1.1**.

**Materiales La Bodega** | *Live Site* **Next.js** | **PostgreSQL** | **Mercado Pago**
*Solo-built e-commerce platform, live in production for a hardware retailer.*

- The only engineer on a live storefront moving about **1.5M COP a day** (roughly USD 11k a month), from requirements and **Mercado Pago** payments to deployment. Built its auth, session security and **RBAC**, hardened against the **OWASP Top 10**.

**PairCode** | *Live Demo* **Node.js** | **Postgres** | **WebSockets**
*Real-time collaborative workspace with an identity layer and realtime transport I wrote myself.*

- I wrote the auth layer instead of importing one: **EdDSA**-signed JWTs, rotating refresh tokens that detect reuse, **Argon2id** hashing and CSRF protection. The custom **WebSocket server** consumes a single-use ticket on handshake and authorizes every event on the server, so a client cannot promote itself by editing a payload.

## Technical Skills

**AI engineering**: spec-driven development with coding agents, Claude Code, Anthropic SDK, Vercel AI SDK, MCP servers, RAG pipelines, structured outputs, agent evals and benchmarks, prompt-injection defense.

**Languages**: TypeScript, JavaScript, Python, C, SQL, Bash.

**Backend**: Node.js, Express, Next.js (App Router, RSC, Server Actions), FastAPI, REST, WebSockets, Prisma, PostgreSQL, MongoDB, Redis.

**Frontend**: React 19, Next.js, Tailwind CSS, React Flow, Liveblocks, Astro, Zod, Jest.

**Architecture & security**: design patterns, Atomic Design, ts-morph static analysis, threat modeling (STRIDE), OWASP Top 10 (Web + LLM), OAuth 2.0 / OIDC, JWT, RBAC, secure code review.

**Cloud & DevOps**: AWS, Azure, Docker, CI/CD (Azure Pipelines, GitHub Actions), Linux, Git.

## Education & Certifications

**Universidad Tecnológica de Pereira**, Pereira, Colombia. Systems & Computing Engineering, in progress (Feb 2022 to present).

**EliteStack Bootcamp**, Pereira, Colombia. Full-Stack Development (Jun 2024 to Jul 2024).

**Certifications**: Google Cybersecurity Certificate, PortSwigger Web Security Academy. In progress: BSCP, Security+.